

Relatório técnico: Sistema de Atendimento por Senha Eletrônica (SASE)

Instituição: Instituto Federal de Educação, Ciência e Tecnologia – Campus Crato

Curso: Bacharelado em Sistemas de Informação

Disciplina: Sistemas Distribuídos

Professor: Guilherme Esmeraldo

Equipe Desenvolvedora: João Pedro & Lucas Oliveira

1. Introdução

Este documento detalha o projeto, a arquitetura e as soluções de software adotadas no desenvolvimento do Sistema de Atendimento por Senha Eletrônica (SASE). O SASE é uma solução de software distribuído projetada para automatizar, gerenciar e otimizar o fluxo de filas em áreas de atendimento ao público.

O sistema foi concebido de forma modular, composto por Terminal de Senhas (TS), Terminal de Atendimento (TA), Terminal de Visualização (TV) e um Servidor central (SRV). Para garantir a escalabilidade e uma melhor experiência de usuário, o projeto evoluiu de uma interface de linha de comando (CLI) para interfaces gráficas interativas (GUI) desenvolvidas com a biblioteca nativa tkinter, incorporando alertas sonoros, geração de relatórios gerenciais e um script de orquestração automatizado para inicialização do ecossistema distribuído.

2. Arquitetura e decisões de design

A arquitetura baseia-se fortemente no modelo cliente-servidor, utilizando *sockets* na camada de transporte (TCP/IP) para garantir a integridade da comunicação entre os nós distribuídos. As principais decisões arquiteturais incluem:

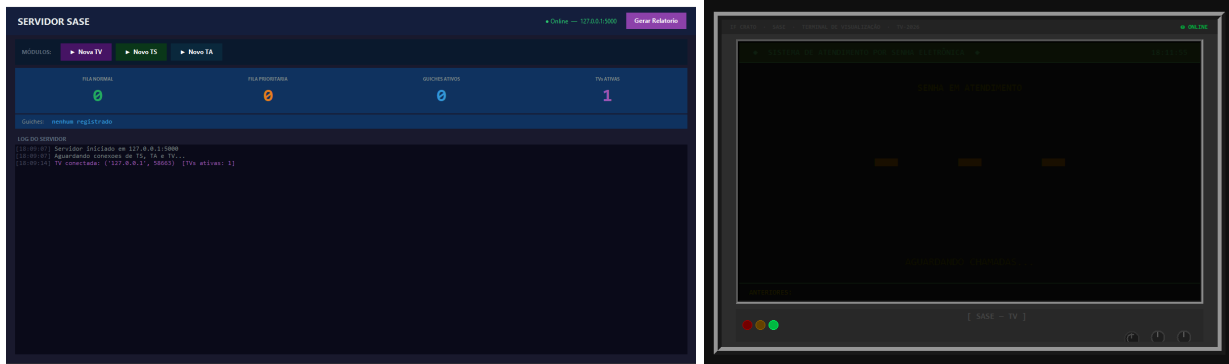
- **Concorrência (Multithreading):** O Servidor (SRV) e as interfaces gráficas (clientes) operam com múltiplas *threads*. No servidor, isso permite aceitar dezenas de conexões simultâneas sem bloqueio de I/O. Nos clientes, as *threads* de rede rodam em paralelo ao *mainloop* do tkinter.
- **Controle de Estado com Mutex:** Como as listas de senhas normais e prioritárias são recursos compartilhados no SRV, utilizou-se mecanismos de travamento (Locks) para aplicar exclusão mútua e prevenir condições de corrida durante o sorteio de senhas.
- **Conexões Persistentes vs. Efêmeras:** O TS utiliza conexões efêmeras para economizar recursos. O TA e o TV estabelecem conexões persistentes com o servidor, permitindo transmissões em tempo real.

3. Especificação dos módulos distribuídos

O sistema foi dividido em quatro processos autônomos com responsabilidades bem definidas.

Módulo	Responsabilidade técnica	Características de rede
Servidor (SRV)	Mediador central. Registra timestamps, gerencia *sockets* ativos e aplica o algoritmo de intercalação estrita (2 Normais para 1 Prioritária).	Socket passivo (Listener). Multithreaded.
Terminal de Senhas (TS)	Totem de autoatendimento. Permite emissão de SEAs do tipo Normal ou Prioritária.	Socket ativo efêmero.
Terminal de Atend. (TA)	Guichê do operador. Requisita senhas, mantém histórico local e gera relatórios em PDF.	Socket ativo persistente.
Terminal de Visual. (TV)	Painel público. Escuta ativamente e exibe a chamada visual e sonora.	Socket ativo persistente (Ouvinte/Broadcast).





4. Estrutura de pastas e módulos auxiliares

A organização do projeto seguiu as melhores práticas de modularização para aplicações Python:

```
projeto_sase/
├── iniciar.py      # Script orquestrador de inicialização
├── servidor/
│   ├── srv.py     # Lógica central e gerenciamento de filas
├── clientes/
│   ├── ts.py      # Interface do Terminal de Senhas
│   ├── ta.py      # Interface do Terminal de Atendimento
│   └── tv.py      # Interface do Pannel de Visualização
├── utils/
│   ├── conexao.py # Configurações de IP e Porta
│   ├── audio.py   # Módulo de integração sonora assíncrona
│   └── relatorio.py # Motor de exportação de dados (PDF/TXT)
```

5. Orquestração e inicialização: Script iniciar.py

Devido à natureza assíncrona dos Sistemas Distribuídos, o servidor deve obrigatoriamente estar em estado de escuta (*listening*) antes que os nós clientes tentem estabelecer conexão. Para automatizar esse processo e melhorar a experiência de *deploy* local, foi desenvolvido o script `iniciar.py`.

Este orquestrador atua nas seguintes frentes:

1. **Validação de Ambiente:** Verifica se a versão do interpretador Python é 3.8 ou superior, garantindo suporte aos recursos utilizados.
2. **Resolução de Dependências:** Inspecciona os pacotes instalados. Caso dependências externas (como `pygame` e `fpdf2`) estejam ausentes, o script invoca o `pip` em sub-rotinas (*subprocesses*) para instalá-las silenciosamente, exibindo indicadores visuais (spinners) ao usuário através de controle de *threads* e sequências ANSI de escape.
3. **Inicialização Ordenada:** Utiliza `subprocess.Popen` para levantar cada nó do sistema em uma janela/processo separado, respeitando um *delay* estratégico: o SRV é iniciado primeiro, seguido da

TV, do TS e, por fim, do TA. Isso mitiga erros de *ConnectionRefused* decorrentes de atrasos na subida da porta TCP do servidor.

6. Protocolo customizado de mensageria

Para o tráfego de dados nos *sockets* brutos, implementou-se um protocolo leve em formato textual, delimitado pelo caractere | e codificado em UTF-8.

- **Emissão de Senha:** TS|GERAR_N ou TS|GERAR_P
- **Registro de Guichê:** TA_CONECTAR|[ID_GUICHE]
- **Solicitação de Chamada:** TA_SOLICITAR|[ID_GUICHE]
- **Registro de Monitoramento:** TV|CONECTAR

7. Instruções de execução

A execução do sistema foi simplificada. A partir do diretório raiz do projeto, basta executar o seguinte comando em um terminal:

```
python iniciar.py
```

Isso abrirá automaticamente as 4 janelas do sistema:

- **[1] SRV — Servidor SASE:** Gerencia as filas e coordena todos os módulos. Abre primeiro e fica em segundo plano.
- **[2] TV — Terminal de Visualização:** Tela de exibição pública das senhas chamadas. Exibe a senha em destaque e toca o som a cada chamada.
- **[3] TS — Terminal de Senhas:** Usado pelo público para gerar senhas Normal (N) ou Prioritária (P).
- **[4] TA — Terminal de Atendimento:** Usado pelo atendente para chamar a próxima senha da fila. Ao abrir, pede o número do guichê.

7.1. Regra de Prioridade

A cada 2 senhas Normais chamadas, a próxima deve ser obrigatoriamente Prioritária (se houver alguma na fila de espera).

7.2. Sistema de Som

O arquivo de áudio deve estar na pasta sons/ ou na raiz do projeto. Formatos suportados: .mp3, .wav, .ogg, .flac.

O som é acionado sempre que:

- Uma senha é gerada no TS;
- Uma senha é chamada no TA;
- Uma senha aparece no TV.